Indoor Positioning from Wi-Fi Fingerprints with Classical Machine Learning on UJIIndoorLoc (Phase 1)

Gauri Mehrotra
Newton School of Technology
Rishihood University
Sonipat, Haryana, India
Enrollment No. 2401010170
gauri.mehrotra2024@nst.rishihood.edu.in

Ansh
Newton School of Technology
Rishihood University
Sonipat, Haryana, India
Enrollment No. 2401010077
ansh.2024@nst.rishihood.edu.in

Agnik Misra
Newton School of Technology
Rishihood University
Sonipat, Haryana, India
Enrollment No. 2401010044
agnik.misra2024@nst.rishihood.edu.in

Kartikey Gupta
Newton School of Technology
Rishihood University
Sonipat, Haryana, India
Enrollment No. 2401010214
kartikey.2024@nst.rishihood.edu.in

Lokendra Singh
Newton School of Technology
Rishihood University
Sonipat, Haryana, India
Enrollment No. 2401010254
lokendra.singh2024@nst.rishihood.edu.in

***Abstract*—GPS fails inside buildings, but the Wi-Fi signals a phone hears show where it is. On the public UJIIndoorLoc data (three buildings, 520 access points), we predict a scan's building, floor and position with classical machine learning. Features, settings and the final model are chosen by user-group cross-validation, which holds out whole users and their phones; the test set, recorded months later mostly with new phones, is used once. Random cross-validation would have promised our Baseline RF, a default random forest, a mean 2D error of 3.67 m; on the test set it reached 12.52 m. Adding our AP-affinity features, which learn where each access point is usually heard, cut its cross-validated error from 15.87 m to 11.38 m. The final model, a tuned Random Forest, finds the building of every test scan and the floor of 92.2 percent, with a mean 2D error of 7.42 m. No model is best at every task (XGBoost finds more floors, 95.2 percent), and our three task models share nothing, which motivates one multi-task network in Phase 2.**

***Index Terms*—Indoor positioning, Wi-Fi fingerprinting, UJIIndoorLoc, random forest, XGBoost, grouped cross-validation.**

I. Introduction

GPS works well outdoors, but inside buildings its signals are too weak to be useful. Wi-Fi is a practical alternative: many buildings already have a Wi-Fi network, so no new hardware is needed [1]. A phone measures the received signal strength (RSSI, in dBm) of every Wi-Fi access point (WAP) it hears: -30 dBm is very strong and -100 dBm is barely heard. One *scan* lists these values, and this list works like a *fingerprint* of the place where it was taken. Fingerprinting records scans at known places and locates a new scan by comparing it with them [2], [3]. Indoor positioning can, for example, guide a visitor to a room on a large campus.

We use the public UJIIndoorLoc dataset [4] and solve three tasks for every scan: predict its *building* (one of three) and its *floor* (one of up to five), two classification tasks, and its *position* (x and y in meters), a regression task. The hard part is testing fairly. Scans recorded in a row at one spot look almost the same, so a random split rewards remembering scans instead of learning places. This is a form of data leakage: information that would not be available in real use reaches the model [5]. This report covers Phase 1, classical machine learning; Phase 2 will build a multi-task neural network. Our contributions are:

- *A fair evaluation.* Features, settings and the final model are chosen by 5-fold user-group cross-validation (CV), which holds out whole users and their phones, and the test set is used only once. We also show how optimistic random CV is.
- *AP-affinity features*, our own idea. They learn from the training scans where each WAP is usually heard and describe a scan by the WAPs it hears. They cut the CV mean 2D error of a default random forest from 15.87 to 11.38 m.
- *A tuned comparison* of kNN, Random Forest and XGBoost, with an error analysis that shows what Phase 2 must improve.

II. Related Work

Fingerprinting. Surveys [1], [3] cover the field. The classic RADAR system [2] located a scan by the recorded scans nearest to it in signal space, the nearest-neighbor rule [6] behind kNN. Horus [7] modeled the variation of the signals

Phase 1 report for Project 9, "Indoor Location Prediction Using Wi-Fi Signals", Team T084, Section D. Code, data and results: `https://github.com/Jitmisra/T084-wifi-indoor-positioning`.

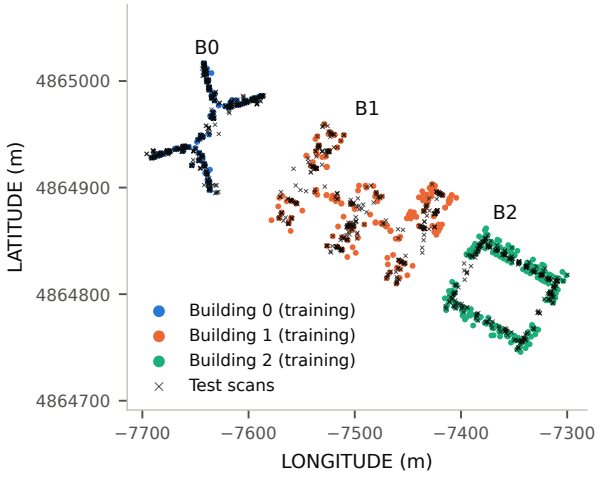


Fig. 1. Where the scans were taken: training positions colored by building and test scans as crosses (Web-Mercator meters). Most test scans lie close to training positions.

with probabilities. Both were tested on the authors' own small testbeds (median error 2.94 m for RADAR, below 0.6 m on average for Horus), so their errors cannot be compared with other studies.

The dataset. UJIIndoorLoc [4] was presented as the first public Wi-Fi fingerprinting database with several buildings and floors. Its validation file was recorded months after the training file, partly by new users and phones, which makes it a realistic test [8]. Torres-Sospedra et al. [8] compared many kNN settings on it, but chose the best one on the file they report, so that result is optimistic.

Tree models. A comparison of classifiers on UJIIndoorLoc found kNN the most suitable, with boosting and bagging bringing a decision tree close to it [9]. We test two tree ensembles, random forests [10] and XGBoost [11], tuned and with our own features.

Neural networks. Nowicki and Wietrzykowski [12] classified building and floor on UJIIndoorLoc with a deep neural network. Kim et al. [13] trained one network with building, floor and location outputs. CNNLoc [14] used a convolutional neural network, but its headline numbers are the best values of different runs, not of one model.

Grouped CV. For grouped data, random CV can seriously underestimate the error, and grouped (block) CV is the better choice [15]. UJIIndoorLoc shows this too: a network that got building and floor right for 98.2% of a random split of the training file did so for only 91.1% of the validation file [12].

Gap. We found no study that compares tuned kNN, random forests and XGBoost on UJIIndoorLoc with every choice made by user-group CV and the validation file used only once, as the test set. Phase 1 fills this gap. Our models still predict building, floor and position separately; multi-task learning [16], which learns related tasks together through a shared representation, is the subject of Phase 2.

III. Data and Exploratory Analysis

UJIIndoorLoc [4] is dataset 310 of the UCI Machine Learning Repository, recorded with smartphones in three buildings of Universitat Jaume I in Spain (Fig. 1). Buildings 0 and 1 have four floors and building 2 has five: 13 building–floor classes. Each row is one scan: the RSSI of 520 WAPs, the building, floor and position, and the user and phone ids. Positions are Web-Mercator meters; one such meter is about 0.77 m on the ground, and we report all errors in these dataset meters.

We train on the file `trainingData.csv`: 19,937 scans recorded from 30 May to 20 June 2013 by 18 users with 16 phones. Our *test set* is the UCI file `validationData.csv`: 1,111 scans recorded from 19 September to 8 October 2013 with 11 phones. It is hard for three reasons: it was recorded about three months after the last training scan; 9 of its 11 phones never appear in training, and they took 64.3% of the test scans; and 55 of the WAPs it hears are never heard in training, so no model can use them.

A scan hears only a few WAPs (17.82 on average in the cleaned training data, 16.48 in the test set), so 96.6% of the training readings are "not heard". The files store "not heard" as $+100$, which would look like the strongest possible signal. We replace it by -110 dBm in both files: weaker than the weakest real reading (-104 dBm), so "not heard" means "very weak".

We clean only the training file; the test file is used exactly as published. In this order, we drop scans that hear no WAP (76), exact duplicate rows (634) and scans with any reading of -30 dBm or stronger (508), which are implausibly strong and come mostly from one phone. This leaves 18,719 training scans. We keep only the 464 WAP columns heard in the cleaned training data, because a WAP that is never heard tells a model nothing. Users who shared a phone form one group, so the 18 training users form 16 user–device groups, which we use for a fair split (Section IV).

IV. Method

A. Fair Evaluation

Because a person records several scans in a row at one spot with one phone, random CV almost always leaves training scans from the same spot, person and phone for a held-out scan. We therefore use 5-fold *user-group cross-validation (CV)* [15], the GroupKFold of scikit-learn [17] with the 16 user–device groups: each fold holds out whole groups, so every model is tested on people and phones that it has not seen, as for most of the test set (9 of its 11 phones are new). Every choice (features, settings, final model) is made with these folds on the training data only, and all that a model learns from data, including the kept WAP columns and the AP-affinity profiles, is learned again in each fold. The test set is used once, at the very end: each model is fitted on all 18,719 training scans and predicts the 1,111 test scans. For comparison only, we also score the Baseline RF with *random CV* (shuffled 5-fold).

B. Features

We compare three feature sets: *rssi*, the RSSI of the kept WAPs (464 columns on the full training data); *rssi+stats*,

which adds three *signal statistics* (the number of WAPs heard, the strongest reading and the mean of the heard readings); and *rssi+stats+affinity*, which adds our eight *AP-affinity* columns.

AP-affinity starts from a simple idea: if a phone hears a WAP that is mostly heard in building 1 on floor 2, the phone is probably there too. It is learned from the training rows only, in two steps. First, every WAP heard in at least five training scans gets a *profile*: the share of its scans in each building (3 values), its mean floor and its mean position (2 values), each a weighted mean over the training scans that hear it. A stronger reading usually means that the phone was closer to the WAP, so it gets a larger weight

$$w(r) = \left(\frac{r - r_{\min}}{-r_{\min}} \right)^{e}, \tag{1}$$

where r is the reading in dBm, $r_{\min}$ is 1 dB below the weakest training reading (-105 dBm on the full training data), $e \approx 2.718$, and $w = 0$ if the WAP is not heard. This "powed" formula [8] gives $w = 0.27$ at -40 dBm, 0.05 at -70 dBm and almost 0 at -100 dBm. Second, a scan s is described by the weighted average of the profiles of the known WAPs that it hears:

$$\mathbf{p}_j = \frac{\sum_i w_{ij}\, \mathbf{t}_i}{\sum_i w_{ij}}, \qquad \mathbf{a}(s) = \frac{\sum_j w_{sj}\, \mathbf{p}_j}{\sum_j w_{sj}}, \tag{2}$$

where $\mathbf{p}_j$ is the profile of WAP j, w_{ij} the weight of WAP j in scan i, and $\mathbf{t}_i$ holds the targets of training scan i (three 0/1 building indicators, floor, x, y). The 6 values of $\mathbf{a}(s)$ and the position of the strongest known WAP in the scan give 8 columns; a scan that hears no known WAP gets the training means.

Missing values and scaling. A WAP that is not heard gets -110 dBm in both files, below every real reading. We add no separate heard/not-heard column per WAP: for a tree, a split below -105 dBm already means "not heard", and `n_heard` counts the heard WAPs. We do not scale the RSSI: trees use only the order of the values, and all RSSI columns share one unit (dBm), so kNN treats them alike. Nothing is ever fitted on the test set. Scaling, fitted on the training data only, is needed in Phase 2, since neural networks train badly on unscaled inputs.

C. Models

Each method predicts building, floor and position with three separate models, one per task: two classifiers and one regressor with two outputs (x, y), built with scikit-learn [17] and the XGBoost library [11].

- *Baseline RF*: a random forest [10] with 300 trees and default settings on rssi+stats.
- *kNN* [6] on rssi: the vote (building, floor) or mean position of the k training scans with the closest RSSI values, plain or weighted by closeness.
- *Random Forest* on rssi+stats+affinity: 300 trees, each trained on a random sample of the scans with a random subset of the features tried at each split; the trees vote or average [10].

TABLE I
TUNED SETTINGS: VALUES TRIED AND THE VALUE CHOSEN BY CV FOR EACH TASK (B: BUILDING, F: FLOOR, P: POSITION)

Setting	Values tried	B	F	P
kNN (28 settings)				
n_neighbors (k)	1, 3, 5, 7, 9, 11, 15	1	5	3
weights	uniform, distance	unif.	unif.	dist.
metric	Euclidean, Manhattan	Eucl.	Manh.	Eucl.
Random Forest (300 trees, 6 settings)				
max_features	sqrt, 0.2, 0.5	sqrt	0.5	0.5
min_samples_leaf	1, 4	1	4	1
XGBoost (6 settings)				
max_depth	4, 6, 10	4	4	10
n_estimators	300, 600	300	300	600

- *XGBoost* on rssi+stats+affinity: trees added one at a time, each correcting the errors of those before it [11]; learning rate (0.1), row sample (0.85) and column sample (0.5) are fixed.

In formulas: kNN compares two scans $\mathbf{r}, \mathbf{r}'$ (RSSI vectors) with the Euclidean or the Manhattan distance,

$$d_2 = \sqrt{\textstyle\sum_j (r_j - r'_j)^2}, \qquad d_1 = \textstyle\sum_j |r_j - r'_j|. \tag{3}$$

A classification tree picks, at each node, the split that most lowers the Gini impurity $G = 1 - \sum_c p_c^2$, where p_c is the share of class c in the node ($G = 0$ when the node is pure); a regression tree lowers the squared error. XGBoost builds its prediction step by step,

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, f_m(\mathbf{x}), \tag{4}$$

where each new tree f_m is fitted to reduce the remaining loss and $\eta = 0.1$ is the learning rate.

D. Tuning and the Final Model

Each model has a small grid (Table I), and every setting is scored on the same five folds. Because each task has its own model, each task picks its own best setting: the highest CV accuracy for building and floor, the lowest CV mean 2D error for position. Before using the test set, we fixed the rule for the final model: the tuned model (kNN, Random Forest or XGBoost) with the lowest CV mean 2D error.

E. Metrics

Building and floor accuracy are the shares of scans with the right building and floor. For the position we report the mean absolute error (MAE) in x and y and the mean and median *2D error*, the straight-line distance between the predicted and the true position: for test scan i with true position (x_i, y_i) and prediction $(\hat{x}_i, \hat{y}_i)$,

$$\mathrm{MAE}_x = \tfrac{1}{N} \textstyle\sum_i |\hat{x}_i - x_i|, \quad e_i = \sqrt{(\hat{x}_i - x_i)^2 + (\hat{y}_i - y_i)^2}. \tag{5}$$

To see where the error comes from, we split the test scans into three cases (building wrong; building right but floor wrong; both right) and give each case's share of scans, mean 2D error and share of the total error. We also compare test scans from seen and new phones.

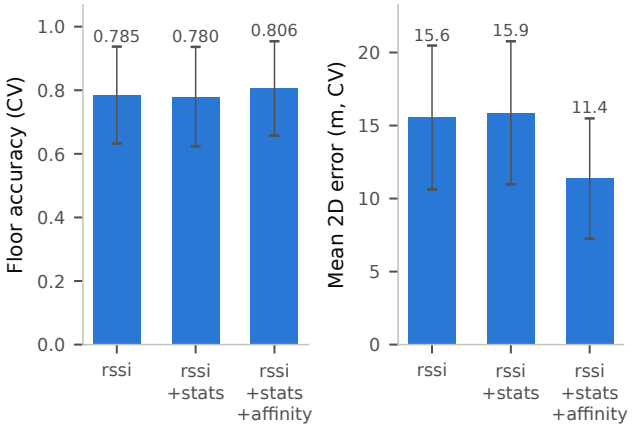


Fig. 2. Feature check: the Baseline RF settings on the three feature sets under user-group CV (bars: mean over the five folds; error bars: standard deviation). Floor accuracy is shown as a fraction.

TABLE II
THE BASELINE RF SCORED IN THREE WAYS

Evaluation	Building (%)	Floor (%)	Mean 2D error (m)
Random CV	100.0	99.9	3.67
User-group CV	99.7	78.0	15.87
Test set	99.8	91.2	12.52

V. RESULTS

A. Feature Check

Fig. 2 shows the Baseline RF settings on the three feature sets under user-group CV. The signal statistics did not help: the mean 2D error rose slightly from 15.55 to 15.87 m and floor accuracy fell from 78.5% to 78.0%. AP-affinity helped clearly: the error fell to 11.38 m and floor accuracy rose to 80.6%. We keep the statistics in the full set because the three sets are nested and the full set scored best; we did not test rssi plus AP-affinity alone. The wide error bars show that the folds differ a lot, as each holds out different people and places.

B. Random CV Is Too Optimistic

For the same Baseline RF, random CV promised 99.9% floor accuracy and a mean 2D error of 3.67 m, but the test set gave 91.2% and 12.52 m (Table II): random CV tests the model on scans that look almost like its training scans (same spot, person and phone). User-group CV (15.87 m) is even stricter than the test set, as a held-out user may be the only one who recorded some places. It errs on the safe side and needs no test data, so we make all choices with it.

C. Model Comparison

Random Forest had the lowest CV mean 2D error (11.09 m, against 11.25 m for XGBoost and 12.75 m for kNN), so by our rule it is the final model. On the test set (Table III) it found the building of every scan and the floor of 92.2%, with a mean 2D error of 7.42 m, against 12.52 m for the Baseline RF. The four models keep the same order by mean 2D error

TABLE III
RESULTS ON THE TEST SET (1,111 SCANS)

Model	Bldg. (%)	Floor (%)	MAE x (m)	MAE y (m)	2D error (m) Mean	2D error (m) Median
Baseline RF	99.8	91.2	8.68	7.40	12.52	7.70
kNN	99.8	87.5	6.12	5.70	9.22	5.88
Random Forest	**100.0**	92.2	**4.77**	**4.64**	**7.42**	**5.34**
XGBoost	**100.0**	**95.2**	5.18	4.77	7.86	6.19

TABLE IV
WHERE THE TEST ERROR COMES FROM

Case	Share of scans (%)	Mean 2D error (m)	Share of total error (%)
Random Forest			
Building wrong	0.0	–	0.0
Building right, floor wrong	7.8	13.06	13.8
Both right	92.2	6.94	86.2
XGBoost			
Building wrong	0.0	–	0.0
Building right, floor wrong	4.8	14.78	9.0
Both right	95.2	7.52	91.0

in CV and on the test set (Fig. 3), so the choice made with CV holds. XGBoost, however, found the most floors (95.2% on the test set, 85.4% in CV): no model is best at every task.

D. Where the Error Comes From

Neither Random Forest nor XGBoost put a test scan in the wrong building; the Baseline RF and kNN did so twice each. For the Random Forest, 87 scans (7.8%) have the right building but the wrong floor, and their mean 2D error (13.06 m) is nearly twice that of the scans with both right (6.94 m; Table IV). Still, 86.2% of the total error comes from the scans with both right: the main remaining problem is the position within the right floor. Of the 87 floor mistakes, 81 are one floor up or down (Fig. 4), and building 1 has the most; for example, 29 of its floor-1 scans were placed on floor 2. XGBoost makes fewer floor mistakes (53) but has a higher error when both are right (7.52 m).

New phones are harder. On the 397 test scans from the 2 phones seen in training, the Random Forest has a mean 2D error of 5.84 m and a floor accuracy of 93.7%; on the 714 scans from the 9 new phones, 8.30 m and 91.3%. All four models are worse on new phones.

VI. PHASE 2 GAP AND CONCLUSION

The gap. Our three task models are separate and share nothing, and the results point to what this misses. First, no model is best at every task: XGBoost finds the most floors (95.2%), Random Forest the best positions (7.42 m). Second, AP-affinity, a hand-made way of sharing location knowledge (where each WAP is heard) with all three tasks, gave our largest gain (CV error from 15.87 to 11.38 m); a model that learns such shared knowledge by itself may gain more. Third, most of the remaining error is position error on the right floor

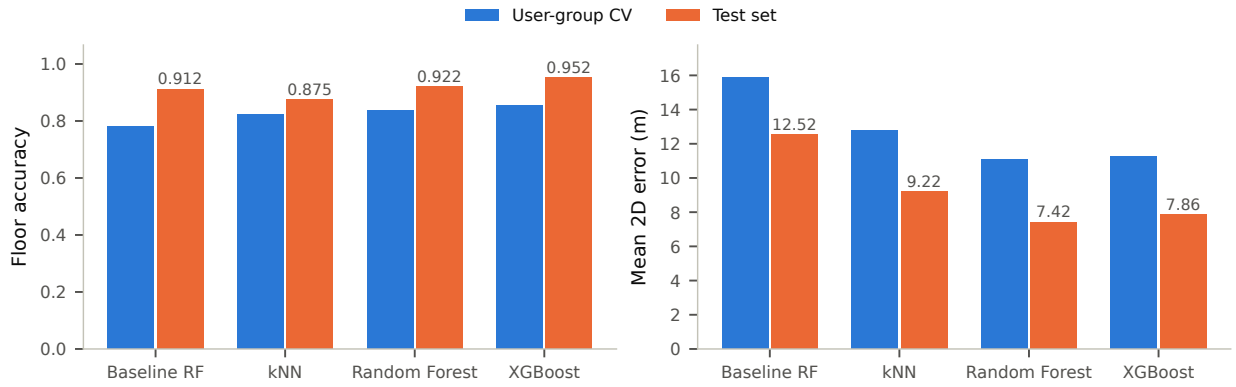


Fig. 3. Floor accuracy (as a fraction) and mean 2D error of the four models in user-group CV and on the test set (test values above the bars).

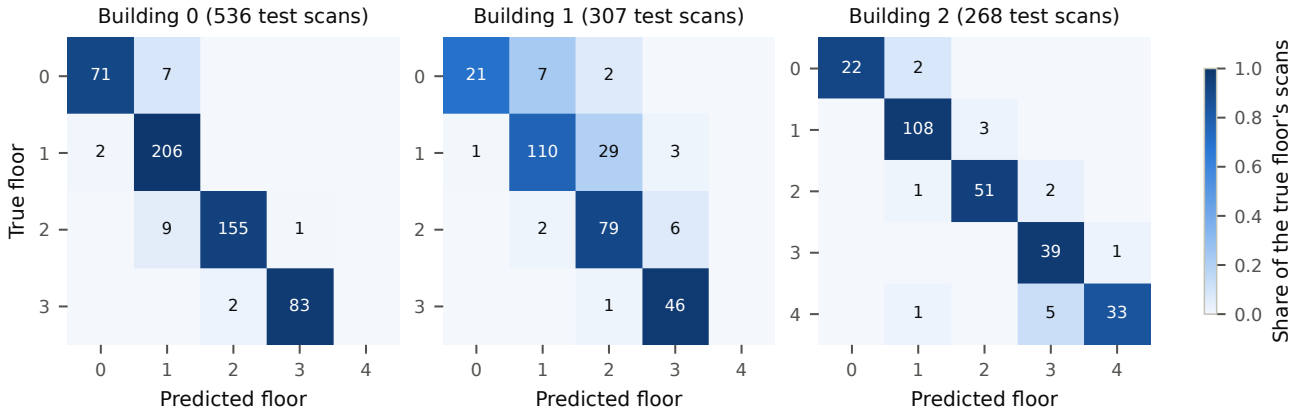


Fig. 4. Floor confusion of the final model (Random Forest) on the test set: test scans per true and predicted floor; colors show the true floor's share.

(86.2% of the Random Forest's total error), and new phones are harder than seen ones (8.30 m against 5.84 m).

Phase 2 will train one multi-task neural network [16] that learns building, floor and position together: shared layers used by all three tasks, then three output heads. Kim et al. [13] showed that one network with building, floor and location outputs can be trained on UJIIndoorLoc. We will evaluate our network with the same user-group CV, test set and metrics, and compare it with our Phase 1 models.

Conclusion. Fair testing matters most: random CV promised 3.67 m, but the Baseline RF reached 12.52 m on the test set. Chosen by user-group CV, the final Random Forest with AP-affinity found every building and 92.2% of the floors, with a mean 2D error of 7.42 m.

Team Contributions

File by file in `CONTRIBUTIONS.md`. Gauri Mehrotra: problem, literature review, report. Ansh: data, cleaning, EDA, fair split (`data.py`). Agnik Misra: features, AP-affinity (`features.py`). Kartikey Gupta: models, tuning, results (`models.py`, `pipeline.py`). Lokendra Singh: metrics, error analysis, Phase 2 gap (`metrics.py`).

References

[1] S. He and S.-H. G. Chan, "Wi-Fi fingerprint-based indoor positioning: Recent advances and comparisons," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 466–490, 2016.

[2] P. Bahl and V. N. Padmanabhan, "RADAR: An in-building RF-based user location and tracking system," in *Proceedings IEEE INFOCOM 2000, Conference on Computer Communications, Nineteenth Annual Joint Conference of the IEEE Computer and Communications Societies*, vol. 2. Tel Aviv, Israel: IEEE, 2000, pp. 775–784.

[3] H. Liu, H. Darabi, P. Banerjee, and J. Liu, "Survey of wireless indoor positioning techniques and systems," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 37, no. 6, pp. 1067–1080, 2007.

[4] J. Torres-Sospedra, R. Montoliu, A. Martínez-Usó, J. P. Avariento, T. J. Arnau, M. Benedito-Bordonau, and J. Huerta, "UJIIndoorLoc: A new multi-building and multi-floor database for WLAN fingerprint-based indoor localization problems," in *2014 International Conference on Indoor Positioning and Indoor Navigation (IPIN)*. Busan, South Korea: IEEE, Oct. 2014, pp. 261–270.

[5] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[6] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[7] M. Youssef and A. Agrawala, "The Horus WLAN location determination system," in *Proceedings of the 3rd International Conference on Mobile Systems, Applications, and Services (MobiSys '05)*. Seattle, WA, USA: ACM, 2005, pp. 205–218.

[8] J. Torres-Sospedra, R. Montoliu, S. Trilles, Ó. Belmonte, and J. Huerta, "Comprehensive analysis of distance and similarity measures for Wi-

Fi fingerprinting indoor positioning systems," *Expert Systems with Applications*, vol. 42, no. 23, pp. 9263–9278, 2015.

[9] S. Bozkurt, G. Elibol, S. Gunal, and U. Yayan, "A comparative study on machine learning algorithms for indoor positioning," in *2015 International Symposium on Innovations in Intelligent SysTems and Applications (INISTA)*. Madrid, Spain: IEEE, 2015, pp. 1–8.

[10] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*. San Francisco, CA, USA: ACM, 2016, pp. 785–794.

[12] M. Nowicki and J. Wietrzykowski, "Low-effort place recognition with WiFi fingerprints using deep learning," in *Automation 2017: Innovations in Automation, Robotics and Measurement Techniques*, ser. Advances in Intelligent Systems and Computing, R. Szewczyk, C. Zieliński, and M. Kaliczyńska, Eds., vol. 550. Cham, Switzerland: Springer, 2017, pp. 575–584.

[13] K. S. Kim, S. Lee, and K. Huang, "A scalable deep neural network architecture for multi-building and multi-floor indoor localization based on Wi-Fi fingerprinting," *Big Data Analytics*, vol. 3, no. 1, 2018, Art. no. 4.

[14] X. Song, X. Fan, C. Xiang, Q. Ye, L. Liu, Z. Wang, X. He, N. Yang, and G. Fang, "A novel convolutional neural network based indoor localization framework with WiFi fingerprinting," *IEEE Access*, vol. 7, pp. 110 698–110 709, 2019.

[15] D. R. Roberts, V. Bahn, S. Ciuti, M. S. Boyce, J. Elith, G. Guillera-Arroita, S. Hauenstein, J. J. Lahoz-Monfort, B. Schröder, W. Thuiller, D. I. Warton, B. A. Wintle, F. Hartig, and C. F. Dormann, "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913–929, 2017.

[16] R. Caruana, "Multitask learning," *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997.

[17] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011. [Online]. Available: https://www.jmlr.org/papers/v12/pedregosa11a.html